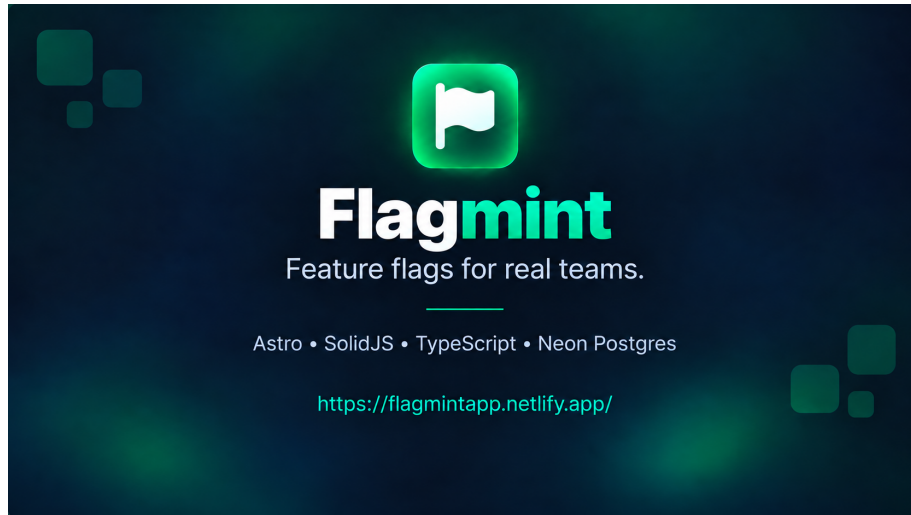




# Flagmint Project Specification v2

Feature flags for real teams.

Implementation profile: Astro + SolidJS + TypeScript + Neon Postgres + Netlify

Flagmint is a lightweight feature-flag service for developers and product teams. It controls feature availability globally, by environment, user, organization or percentage rollout. Deterministic evaluation and secure API-key handling are the principal technical requirements.

## Purpose of this document

This specification translates the product PRD into an implementation plan. It defines the application architecture, database shape, server interfaces, UI modules, service integrations, testing strategy, deployment model and delivery criteria.

# 1. Product and Functional Scope

## User roles

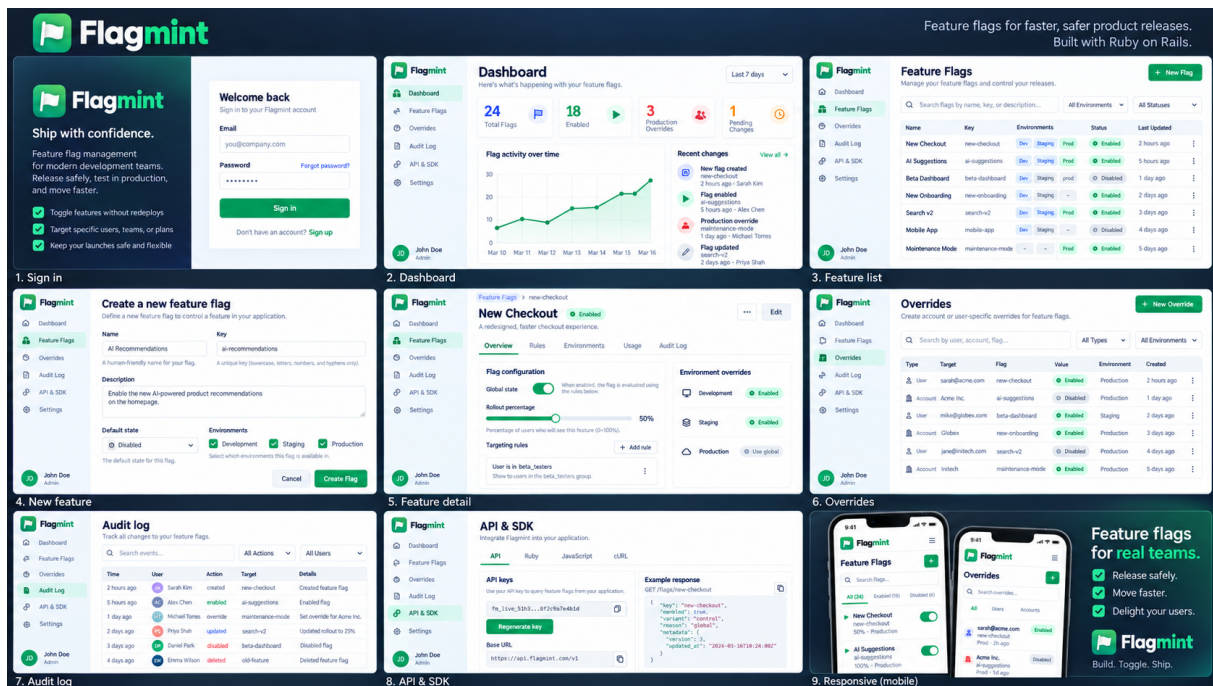
|                               |                                                                       |
|-------------------------------|-----------------------------------------------------------------------|
| <b>Developer/Product User</b> | Create flags, configure environments/rules and inspect audit history. |
| <b>Administrator</b>          | Manage users, API keys and workspace settings.                        |

## Required screens

- Sign in
- Dashboard
- Feature list
- New feature
- Feature detail
- Overrides
- Audit log
- API & SDK
- Settings
- Mobile views

## Scope discipline

Build the primary workflow first. Optional analytics, advanced filters and secondary settings should only be added after the core workflow, permissions, tests and deployment are stable.



## 2. System Architecture

### Target architecture

Browser -> Cloudflare DNS -> Netlify -> Astro server routes/pages -> SolidJS islands -> TypeScript service layer -> parameterized SQL -> Neon Postgres. Neon Auth provides authentication. R2, Resend and other services are added only where the project requires them.

### Architecture principles

- Use Astro for routing, SSR, API/server endpoints and page composition.
- Use SolidJS only for interaction-heavy islands rather than turning the product into an unnecessary SPA.
- Keep server-only code outside the client bundle.
- Use direct parameterized SQL; centralize queries by domain to avoid SQL scattered through UI components.
- Validate every external mutation with Zod on the server.
- Authorization is enforced server-side even when controls are hidden in the UI.
- Keep infrastructure minimal; do not add services without a concrete product need.

### Repository structure

```
src/ components/ layouts/ pages/ pages/api/ lib/ auth/ db/ validation/ services/ features/ styles/ tests/  
unit/ e2e/ public/ netlify/ functions/ (only where needed) .github/workflows/
```

## 3. Technology Stack

|                              |                                                                           |
|------------------------------|---------------------------------------------------------------------------|
| <b>Application framework</b> | Astro                                                                     |
| <b>Interactive UI</b>        | SolidJS                                                                   |
| <b>Language</b>              | TypeScript                                                                |
| <b>Build tooling</b>         | Vite (through Astro)                                                      |
| <b>Runtime</b>               | Node.js                                                                   |
| <b>Styling</b>               | Tailwind CSS                                                              |
| <b>Database</b>              | Neon Postgres                                                             |
| <b>Authentication</b>        | Neon Auth                                                                 |
| <b>Database access</b>       | Direct parameterized SQL - no ORM                                         |
| <b>Validation</b>            | Zod                                                                       |
| <b>Object storage</b>        | Cloudflare R2 where files are required                                    |
| <b>Email</b>                 | Resend where transactional mail is required                               |
| <b>Testing</b>               | Vitest + Playwright                                                       |
| <b>Code quality</b>          | ESLint + Prettier                                                         |
| <b>Monitoring</b>            | Sentry                                                                    |
| <b>CI/CD</b>                 | GitHub Actions                                                            |
| <b>Hosting</b>               | Netlify                                                                   |
| <b>DNS / edge</b>            | Cloudflare DNS                                                            |
| <b>TLS</b>                   | Netlify-managed certificate; Cloudflare proxy optional with Full (strict) |

### Node.js usage

Node.js is the server/runtime and tooling environment. Pin a current LTS version in the repository so local development, CI and Netlify use a consistent runtime.

### Direct SQL conventions

- Use the Neon serverless/Postgres driver appropriate for the chosen Astro runtime.
- All values are passed as parameters; never construct SQL with untrusted string interpolation.
- Keep migrations as versioned SQL files in the repository.
- Wrap multi-step state changes in database transactions.
- Create indexes for foreign keys and fields used heavily by filtering/search.

## 4. Data Model

### Core tables

|                          |                                                                                                       |
|--------------------------|-------------------------------------------------------------------------------------------------------|
| <b>users</b>             | id, email, name, role                                                                                 |
| <b>feature_flags</b>     | id, key, name, description, default_state, created_by, created_at                                     |
| <b>environments</b>      | id, key, name                                                                                         |
| <b>feature_rules</b>     | id, feature_flag_id, environment_id, target_type, target_value, enabled, rollout_percentage, priority |
| <b>feature_overrides</b> | id, feature_flag_id, environment_id, subject_type, subject_id, enabled                                |
| <b>api_keys</b>          | id, name, environment_id, key_prefix, key_hash, revoked_at                                            |
| <b>audit_events</b>      | id, actor_id, feature_flag_id, action, metadata_json, created_at                                      |

### Migration strategy

Store ordered SQL migration files in source control. Each migration must be forward-only for the portfolio MVP. Seed scripts should create realistic demo data without embedding production credentials.

### Data integrity

- Use NOT NULL constraints for required fields.
- Use foreign keys for relational ownership.
- Use CHECK constraints for stable enums/statuses where practical.
- Use unique constraints for natural keys such as API keys, feature keys or emails where applicable.
- Use timestamptz for timestamps.

## 5. Server Routes and UI Modules

### Primary server interfaces

- CRUD /flags
- POST /flags/:id/toggle
- CRUD /flags/:id/rules
- CRUD /flags/:id/overrides
- GET /api/v1/flags/:key - evaluate flag
- POST /api-keys - create key
- DELETE /api-keys/:id - revoke key
- GET /audit

### Primary UI components / Solid islands

- FlagTable
- FlagToggle
- EnvironmentTabs
- RuleEditor
- OverrideTable
- EvaluationTester
- ApiKeyList
- AuditTimeline

### Implementation pattern

Pages should render useful content on the server. Solid islands handle stateful widgets such as filters, forms, calendars, progress displays, inspectors and interactive tables. Server mutations should return structured success/error results that map cleanly to form states.

## 6. Business Rules, Security and Integrations

### Project-specific rules

- Evaluation precedence: user override -> organization override -> environment rule -> global/default.
- Percentage rollouts use deterministic hashing so a subject receives a stable result.
- API keys are shown once and stored only as hashes.
- Flag keys are unique and immutable after first use unless an explicit migration path is provided.
- Every configuration mutation creates an audit event.

### External services

|                          |                                      |
|--------------------------|--------------------------------------|
| <b>Neon Auth</b>         | Identity.                            |
| <b>Neon Postgres</b>     | Flags, rules, keys and audit events. |
| <b>Sentry</b>            | API and UI error monitoring.         |
| <b>Netlify Functions</b> | Flag evaluation API.                 |

### Security baseline

- Rely on Neon Auth/session state server-side for identity.
- Validate authorization on every protected server route.
- Use Zod for request payload validation.
- Use secure, HTTP-only cookies according to the Neon Auth integration.
- Rate-limit abuse-sensitive public endpoints where practical.
- Never expose database, R2, Resend or Sentry server secrets in browser code.
- Escape/sanitize user-controlled text before rendering.
- Log errors to Sentry without logging secrets or sensitive payloads.

## 7. Testing and Quality

- Vitest: rule precedence, deterministic rollout, key hashing and evaluation edge cases.
- Playwright: create flag, toggle, add override, create API key, verify audit log.
- API integration: authenticated evaluation returns correct result.

### CI pipeline

On each pull request / push: 1. npm ci 2. TypeScript typecheck 3. ESLint 4. Prettier check 5. Vitest 6. Build Astro application 7. Playwright on selected branches or against preview deployment

### Quality gate

A feature is not considered complete until its server-side validation, authorization path and primary failure case have automated coverage.



## 8. Deployment and Operations

### Netlify

- Connect the GitHub repository to Netlify.
- Use deploy previews for pull requests.
- Configure Astro's Netlify adapter/runtime as required by the final app mode.
- Store all secrets in Netlify environment variables.
- Use Netlify Functions/Background Functions only where the product requires them.
- Expose a lightweight health route where useful.

### Cloudflare DNS / TLS

Point the chosen custom domain/subdomain to Netlify through Cloudflare DNS. Netlify can provision the origin/site certificate. If Cloudflare proxying is enabled, use Full (strict) so Cloudflare validates the Netlify certificate.

### Environment variables

- DATABASE\_URL
- NEON\_AUTH\_\*
- SENTRY\_DSN
- FLAG\_API\_RATE\_LIMIT

### Observability

Configure Sentry for production and preview environments. Capture unhandled server errors and meaningful client failures. Add release/environment tags so a deployed error can be traced to a commit.

## 9. Implementation Sequence

- 1. Bootstrap repository: Astro, Solid, TypeScript, Tailwind, ESLint, Prettier, Vitest and Playwright.
- 2. Configure Neon Postgres and versioned SQL migrations.
- 3. Integrate Neon Auth and implement authorization helpers.
- 4. Build the core data model and seed/demo data.
- 5. Implement one complete end-to-end workflow before secondary screens.
- 6. Add project-specific external services such as R2 or Resend.
- 7. Add Solid islands only where interaction materially benefits from them.
- 8. Add Vitest coverage for domain logic and Playwright coverage for primary user journeys.
- 9. Configure GitHub Actions, Netlify previews and Sentry.
- 10. Polish README, screenshots, demo account/data and known limitations.

## Definition of Done

- Flags can be created and toggled.
- Environment and subject overrides work.
- Evaluation API is deterministic.
- API keys are secure.
- Audit history is complete.
- Tests pass.
- Netlify deployment is healthy.

### Portfolio requirement

The repository README should include the project banner/logo, one-sentence value proposition, live demo link, screenshots, feature list, architecture summary, tech stack, local setup, environment-variable template, test commands, known limitations and license.